Management Decision Science (MDS)

Queuing Theory and the Mess Queue Simulation

How waiting-line models, and the other MDS course topics, apply to the Chefs Touch lunch problem and to our prototype

Project: Beyond the Bottleneck: Enabling Smarter Lunch Flow by Optimising Queue Management and Capacity Utilisation Across Chefs Touch Centres

Phase 1 title: The Two-Hour Bottleneck: Optimising Critical Commodity Allocation Across CT's Centres During Peak Lunch Hours

Group: WGB3

Focus centres: Central CT (Chefs Touch) and Munch Mess, lunch 11:30–14:30

Date: October 2026 · Final version (corrected calculations; see Appendix C)

> **How to read the numbers.** Every figure in this document is tagged. **Measured** = from our field work or from CT. **Assumed** = a placeholder until we measure it. **Illustrative** = a worked example to explain a method, not a finding. The prototype currently runs on simulated data only.

Contents

1. The problem in brief (and where it sits in Operations Management)
2. Queuing theory: concepts and notation
3. The core formulas, with worked examples
4. Where the textbook model breaks at a mess, and what we do instead
5. How the simulation applies queuing theory
6. Results from the simulation
7. The other MDS course topics applied to the project
8. What we have measured and what we still assume
9. Appendix A: formula sheet
10. Appendix B: where each idea lives in the code
11. Appendix C: what changed in this final version

1. The problem in brief

Chefs Touch (CT) cooks lunch in a central kitchen and supplies five centres: Central CT, AB2, AB3, AB4 and New Mess. Lunch (11:30–14:30) is the busiest three hours on campus. Demand is concentrated into a short window while kitchen, delivery and serving capacity are fixed, so small mismatches turn into queues, run-outs and waste.

1.0 From Phase 1 to Phase 2: the revised problem

Phase 1 (*The Two-Hour Bottleneck*) asked whether CT's **food** was reaching the right centre, in the right quantity, at the right time. The field visit answered part of that, and it showed that food is only half the problem. Students also wait because of **how they and the serving capacity are matched**: most go to Central CT by habit and overload its two serving stations, while Munch Mess, serving the same menu, runs well below capacity. When a star dish runs out at the busy mess, its queue gets longer still.

Revised problem statement (Phase 2). During the 11:30–14:30 lunch window, Chefs Touch serves about 2,960 students. Total serving capacity is enough on average, but it is used unevenly: one centre is saturated while another has spare capacity, so students wait (about 15 minutes observed) even though the system as a whole could serve them faster. Food run-outs at the busy centre make this worse. **The project uses biometric entry scans, queuing theory and live screens to balance student flow across centres and to give the mess manager earlier signals about food, so the same capacity serves the same students with less waiting.**

Decision question: how should student flow and serving capacity be balanced across CT's centres, minute by minute, so that waiting falls without adding staff or space?

	Phase 1: The Two-Hour Bottleneck	Phase 2: Beyond the Bottleneck
Focus	Allocating critical food items across five centres	Managing queues and using capacity evenly, with food as an input
Main lever	How much food goes where, and when	Where students eat (live screens), plus earlier food signals from the manager
Evidence	Field visit at New Mess, CT interviews, item-wise risk (ABC/XYZ)	The same, plus biometric entry scans (requested) and a working prototype on simulated data
Scope now	Five centres	Central CT and Munch Mess (they serve the same menu side by side); the method extends to all five
Methods	Forecasting, weighted decision table	Queuing theory, simulation, routing, LP/IP, decision analysis, MCDM

Food allocation stays in the project: the priority matrix sets how much each tray matters to the live advice (section 7.4), and the LP allocation and decision tree (sections 7.5–7.6) still decide how much food to send and when.

Phase 1 evidence (measured, from our deck and field visit on Tuesday 1 September):

- About **2,960 plates** served at lunch, against a planned range of **2,810–3,465**.
- Replenishment starts only when the mess calls the kitchen because a dish is **running low**. If the kitchen has to make it, the refill arrives **30 min** after the call (making plus sending). If it is already made, only sending is needed: **10–14 min**. Either way, top-ups arrive inside the peak.
- Students faced delays of about **15 min**. Star dishes ran out by **13:20–13:23**, with more than half of service still to go.

- Seating: **Central CT ≈ 900**, **Munch Mess ≈ 492** (together ≈ 1,392, matching the deck's 1,400–1,600).
- A normal visit, from biometric scan to leaving, takes about **18 minutes**. Queueing during surges makes it longer.
- Each mess has **2 serving stations** and **1 face scanner** at the entrance (measured). Field notes: the queue builds **at the serving stations**; the single scanner can also become a bottleneck at peak (section 6).
- All **2,960 students** (one plate each) eat at Central CT or Munch Mess. By habit about **2,360 eat at Central and ~600 at Munch** (80/20, the group's count).

This is a **waiting-line problem** (how long students wait and why) inside an **allocation problem** (where food and students should go). Queuing theory explains the first part; section 7 shows how the other course topics handle the second.

1.1 Where the problem sits in Operations Management

The problem falls mainly under **capacity management** and **queuing theory**, two core parts of Operations Management (OM). Because students choose where to eat, it also falls under **behavioural operations** and **service operations**. We checked each area against our own numbers before using it:

Area of OM	What it means	How it applies here, with our numbers	Where we use it
1. Capacity management and load balancing	Matching resources (seats, food, staff) to demand	Total capacity is enough, but the load is uneven. In the busiest 10 minutes (13:10, when several departments' lunch breaks overlap) about 37 students/min arrive in all, against 40/min of serving capacity (2 stations × 2 messes × 10/min, μ assumed), so ρ ≈ 0.93 overall. With 80% going to Central by habit (measured: ~2,360 of ~2,960), Central gets ≈ 29/min against 20/min (ρ ≈ 1.46, saturated, and close to its single scanner's ~30/min limit) while Munch gets ≈ 7.5/min against 20/min (ρ ≈ 0.38). Seats are never the constraint: the stations let at most 20 students a minute through, so at most ≈ 320 seats per mess are in use (section 3.2)	Equilibrium split and door screens (5.3); integer staffing per slot (7.7); LP allocation of food between messes (7.6)
2. Queuing theory (wait-time management)	Why lines form: arrival rate, number of servers, service rate	The servers are the 2 serving stations per mess, with the single face scanner as a first phase. At saturation, **throughput is capped** at cμ = 20/min and the queue grows by λ − cμ each minute. Throughput actually **drops** during stock-outs and while staff refill wells over the counter (field note O4)	Sections 2–6
3. Behavioural operations	How human behaviour shapes an operational system	Habit (80/20, measured by the group), herding if every screen says 'go to the emptier mess', partial compliance (κ = 60%, assumed), and the star dish that students come for. The **reasons** for the habit (food, distance from hostels, friends) are not measured yet: our survey and the biometric logs will show them	Compliance κ and anti-herding split (5.3); biases (7.3); student-screen design (see docs/student-screen-psychology.md)
4. Demand shaping	Influencing **where** and **when** demand	Our lever is **information**: live waits on screens outside each mess and on phones,	Door screens and phone page; AHP

Area of OM	What it means	How it applies here, with our numbers	Where we use it
	arrives instead of adding capacity	plus a 'fresh batch just out' cue. Shaping **when** students arrive (staggered class end times) is alternative A5 in the AHP. Other nudges (a dish only at Munch, loyalty points, grab-and-go) are possible future options; we have not built or tested them	alternatives (7.8)

What we changed from the general framing. (1) We call area 4 **demand shaping**, not yield management: yield (revenue) management sets **prices** for perishable capacity, and a prepaid mess has no price lever. (2) 'Cashiers' become **serving stations** and the **face scanner**. (3) 'Throughput slows at saturation' is sharpened: throughput is capped at capacity, and it falls during stock-outs. (4) The reasons students prefer Central are **hypotheses**, not findings, until the survey and logs are in.

2. Queuing theory: concepts and notation

2.1 What queuing theory is for

Queuing theory is the mathematical study of waiting lines. It models **arrivals**, **waiting time**, **service capacity**, **queue length** and **server utilisation**. Its goal is **not** to eliminate waiting. It is to find an acceptable balance between how long customers wait and how much service capacity costs.

It helps a manager answer four questions, each of which applies directly to a mess:

- How much service capacity is needed? (How many serving stations at peak?)
- How long will customers wait? (What should the door screen say?)
- When should another counter open? (At what arrival rate?)
- How can waiting be cut without a high cost? (Spread students between messes, stagger arrivals, pre-stage food.)

2.2 Why queues form

A queue forms whenever **demand for service temporarily exceeds the capacity available**. At a mess, all the textbook causes are present:

Cause (from Session 13)	What it looks like at the mess
Customers arrive randomly	Students arrive at irregular times (for example, classes ending at different times, or friends arriving together). The biometric logs will show the actual pattern
Service times differ	Some students take more items or wait for a refill
Demand is concentrated in a peak	About 2,960 plates are served within the 11:30–14:30 window (measured). Our field visit saw a full hall and a long queue around 13:18–13:23 (observed). Department lunch breaks overlap (group's timetable): MIT and SMI break at 12:00, DOC/MICFE at 12:30, DLHS/MILHS at 12:40, MLS at 12:50, MIT's lab-day batch at 13:00 and TAPMI at 13:05, so the heaviest overlap is 13:05–13:30 (section 5.1). The biometric logs will confirm the real curve
Limited capacity	2 serving stations and 1 face scanner per mess (measured)
Unexpected disruptions	A star dish runs out; staff refill the well at the counter while the line waits

> **Key insight from the lecture:** even when **average** capacity is enough, **variation** still creates queues. For example (hypothetically), a mess that could serve everyone if they arrived evenly over two hours would still build a queue if half of them arrived within the same 30 minutes.

2.3 Anatomy of a queuing system, mapped to the mess

Component	Definition	At Central CT / Munch Mess	Data source
1. Calling population	Who needs service	Students (and some staff/faculty) eating lunch	~2,960 plates per lunch (measured, Phase 1)
2. Arrival process	How often customers arrive	Students scanning in at the entrance	Biometric logs: exact time of every arrival
3. Waiting line	Where customers wait before service	The line at the serving counter	Field observation; estimated live by the prototype
4. Service	The servers	Staff serving food at each serving	Not yet measured: needs a

Component	Definition	At Central CT / Munch Mess	Data source
mechanism		station	stopwatch
5. Queue discipline	Rule for who is served next	First come, first served (FCFS)	Observed
6. Exit	Customer leaves after service	Student eats (14–18 min), disposes of the plate (1–2 min) and leaves	Total 17–35 min door to exit (group estimate); ~20 min on a normal day

2.4 Types of waiting-line structure

Structure	Meaning	Mess example
Single channel, single phase	One queue, one service point	A single serving station with one server
Multiple channel, single phase	Several servers doing the same job	Several identical serving stations in one hall
Single channel, multiple phase	Customers pass through stages in order	Tray counter → food line → seat, with one food line (New Mess)
Multiple channel, multiple phase	Several servers at several stages	Face scanner (1) → serving stations (2) → seat: Central CT and Munch Mess

A mess is **multiple-phase**: students pass the face scanner (1 per mess), then a serving station (2 per mess), then a seat. In a multi-phase system the **slowest phase is the bottleneck** and sets the wait. Our field photos (queue at the food line, free seats) point to the **serving stations**, so they are the main service mechanism. The simulator also models the scanner as its own phase (an assumed 2 s per scan, so about 30 scans/min), because at peak it comes close to its limit.

2.5 Notation: every variable used in this document

Standard queuing symbols (from the lecture):

Symbol	Name	Meaning	Units	Mess value / status
λ (lambda)	Arrival rate	Average number of students arriving per unit time	students/min or /hour	From biometric logs, per 10-min bin. Not yet received
μ (mu)	Service rate	Average number of students ONE serving station can serve per unit time	students/min per station	10/min (≈6 s each), **assumed**
c	Number of servers	Serving stations working in parallel	count	2 per mess, **measured**
cμ	Capacity	Total students the hall can serve per unit time	students/min	20/min per mess (2 × 10)
ρ (rho)	Utilisation	Share of serving capacity in use: ρ = λ / (cμ)	fraction (0–1)	Computed
L	Number in system	Average students in the whole system (queueing + being served + eating)	students	Computed (Little's Law)
Lq	Number in queue	Average students waiting before service	students	Computed
W	Time in system	Average total time from arrival to leaving	minutes	~18 min on a normal day, **measured**

Symbol	Name	Meaning	Units	Mess value / status
Wq	Time in queue	Average waiting time before service starts	minutes	Computed
1/μ	Service time	Average time to serve one student	minutes	0.1 min (6 s), **assumed**

Additional variables used in our simulation and prototype:

Symbol	Name	Meaning	Value / status
a = λ/μ	Offered load	Workload in 'server-equivalents'; used inside the M/M/c formula	Computed
C(c, a)	Erlang C	Probability an arriving student has to wait at all (M/M/c)	Computed
t, Δt	Time, time step	The simulation and the live system update every Δt = 5 s (the scanner poll interval)	5 s
B(t)	Backlog	Estimated students waiting to be served at time t (the live queue estimate)	Estimated from scans
A(t)	Arrivals in a step	Number of scans between t and t + Δt	Measured (scans)
T_eat	Eating time	Average time a student spends seated after being served	14–18 min (mean 16), group estimate
S(t)	Seated	Students served in the last T_eat minutes (still eating)	Estimated
$\hat{L}(t)$	People inside	Estimate of L at time t: $\hat{L}$ = B(t) + S(t)	Estimated
N	Seats	Seating capacity of a mess	Central 900, Munch 492, **measured**
H	Horizon	How far ahead a prediction must hold (students arriving in the next H minutes)	5 min
s_walk	Walk time	Walk between the two mess doors	30 s, **assumed**
cv	Coefficient of variation	Spread of service times relative to their mean (1 = exponential, as M/M/1 assumes)	0.6, **assumed**
h_i	Habit share	Share of students who go to mess i by habit, without advice	80% Central / 20% Munch (~2,360 / ~600), **measured** by the group
p_i	Target share	Share of students the system wants in mess i	Computed
s	Scanners	Face scanners at the entrance of each mess, one at a time	1 per mess, **measured**; 2 s per scan, **assumed**
κ (kappa)	Compliance	Share of students who follow the screen's advice	60%, **assumed**
Δ_min	Minimum saving	Students are only asked to switch if it saves at least this much time	2 min

3. The core formulas, with worked examples

3.1 Utilisation and stability

$$\rho = \lambda / \mu \quad \text{(one server)} \qquad \rho = \lambda / (c \cdot \mu) \quad \text{(c servers)} \quad \text{(utilisation)}$$

ρ is the share of time the servers are busy. The lecture's example: λ = 18 students/hour and μ = 24/hour at Hatti Kaapi give ρ = 0.75, so the barista is busy 75% of the time.

Stability. A queue is **stable** if λ < cμ: the servers can eventually clear it. It is **unstable** if λ > cμ: students arrive faster than they can be served and the queue keeps growing.

At the mess: the lunch peak is **temporarily unstable**. For part of the peak, arrivals can exceed serving capacity, and the queue then grows by roughly (λ − cμ) students every minute. Our observations (long queue, run-outs around 13:20) suggest this happens; the biometric logs will show when it starts and how long it lasts. When demand repeatedly exceeds capacity, waiting becomes a structural problem rather than a temporary inconvenience. That is what Phase 1 called the 'Two-Hour Bottleneck'.

3.2 Little's Law

$$L = \lambda \cdot W \quad \text{(Little's Law)}$$

The average number of people in a system equals the arrival rate multiplied by the average time each spends there. It holds for almost any queue and needs no assumptions about arrival or service patterns, which makes it the most useful formula for us.

Application 1: can the seats cope? (measured inputs)

Rearranged, λ_max = N / W gives the highest arrival rate the seats can sustain. A seat is held only while eating (14–18 min, group estimate), so W here is the eating time, not the whole visit:

Mess	Seats N (measured)	Seat held W (eating, mean)	λ_max = N / W
Central CT	900	16 min	**≈ 56 students/min**
Munch Mess	492	16 min	**≈ 31 students/min**
Together	1,392	16 min	≈ 87/min

The seats can absorb far more than the ~2,960 plates served in the whole lunch. **Seats are not the bottleneck; serving speed is.** This matches our field notes and the simulation.

Application 2: why the seats never fill (from our stage timings)

Little's Law applies to each stage on its own. A seat is used only by a student who has been served, and the serving stations serve at most cμ = 20 students a minute. So, however long the queue, seats in use ≤ 20/min × 16 min of eating ≈ 320 at each mess: about a third of Central's 900 and two-thirds of Munch's 492. The simulation agrees (peak 341 seats in use at Central, 38%). A hall can still look full, as in photo O1 at 13:18, because the people standing in the queue are inside too: L ≈ 29/min × 26 min ≈ 750 people in the hall in all, most of them queueing, not seated. The constraint is the serving stations, not the seats.

Application 3: checking our data

When the biometric logs arrive, we compute L = λ × W for each 10-minute slot (W = the throughput time, 17–35 min) and compare it with a headcount or photo. A large mismatch tells us either the stage times or the data need a second look.

3.3 The M/M/1 model (one serving station)

M/M/1 means: random (Poisson) arrivals, random (exponential) service times, **1** server, first come first served, unlimited waiting space, and $\lambda < \mu$.

$$Wq = \lambda / (\mu\,(\mu - \lambda)) \qquad W = 1 / (\mu - \lambda) \qquad Lq = \lambda \cdot Wq \qquad L = \lambda \cdot W \quad \text{(M/M/1)}$$

Lecture example: Hatti Kaapi

λ = 18/hour, μ = 24/hour → ρ = 75%; Wq = 18 / (24 × 6) = 0.125 h = **7.5 min**; W = 1/6 h = **10 min**; Lq = 18 × 0.125 = **2.25** students waiting.

Mess example (illustrative: one serving station, μ = 10 students/min, i.e. 6 s each, as everywhere else in this document)

With λ = 8.33/min (500/hour): ρ = 0.833; Wq = λ / ($\mu(\mu - \lambda)$) = 8.33 / (10 × 1.67) = 0.5 min; W = 1 / ($\mu - \lambda$) = 0.6 min; Lq = λ·Wq ≈ 4.2 students waiting; L = λ·W = 5.

Why waits explode near full utilisation (illustrative, one station, μ = 10/min)

Utilisation ρ	75%	83%	90%	96%	98%
Wq, average wait	0.3 min	0.5 min	0.9 min	2.4 min	4.9 min
Lq, average queue	2.3	4.2	8.1	23	48

Going from 90% to 98% busy multiplies the wait by more than five. As the lecture puts it, **maximum utilisation does not mean maximum efficiency**.

Reading the observed 15-minute delay backwards

A steady-state formula cannot produce the 15-minute delay we observed. At μ = 10/min even 98% busy gives a 4.9-minute wait on one station and about 2.4 minutes with Central's two (M/M/c, section 3.4). A 15-minute wait means the line was over capacity for a while and built up, which is the fluid queue of section 4.1: 15 minutes at 20 served a minute is about 15 × 20 = 300 people ahead. At ~30 arrivals/min against 20 served, the line grows by 10 a minute, so that takes about 30 minutes of surge: the 13:05–13:35 overlap of department lunch breaks. A food run-out makes it worse, because the stations serve nobody until the refill.

3.4 The M/M/c model (several serving stations)

When a hall has c identical serving stations fed from one queue, we use **M/M/c**. The Erlang C formula gives the probability that an arriving student has to wait at all:

$$a = \lambda/\mu, \quad C(c, a) = [\ (a^c / c!) \cdot c/(c - a)\ ] \,/\, [\ \Sigma_{k=0}^{c-1}\ a^k / k!\ +\ (a^c / c!) \cdot c/(c - a)\ ] \quad \text{(Erlang C)}$$

$$Wq = C(c, a) / (c\cdot\mu - \lambda) \qquad W = Wq + 1/\mu \qquad Lq = \lambda\cdot Wq \quad \text{(M/M/c)}$$

Example with our mess inputs: c = 2 serving stations (measured), μ = 10 students/min per station (assumed), so capacity 20/min:

Arrivals λ	Utilisation ρ	P(wait) C	Wq	Lq (waiting)
15/min	75%	0.64	0.13 min (8 s)	1.9
17/min	85%	0.78	0.26 min (16 s)	4.4
19/min	95%	0.93	0.93 min (56 s)	17.6
24/min	120%	—	unstable: queue grows by 4 students/min	growing

Two stations pooled into one queue wait much less than two separate M/M/1 lines at the same utilisation. That is the case for one shared queue feeding both serving stations. Why μ = 10/min: with only 2 stations, Central must serve about 2,360 students (80% of 2,960), mostly within two hours of breaks, which is impossible at slower speeds; it must be about this fast. A stopwatch check is the most valuable next measurement.

3.5 Capacity management: design capacity, effective capacity and the peak

Capacity management uses four standard measures. Design capacity is the most a process can produce: for a mess, the bottleneck's rate (2 serving stations × 10/min = 20/min) over the 180-minute lunch, 3,600 students. Effective capacity takes off expected losses, here the minutes the stations stand idle while a star dish is out. Utilisation = actual output ÷ design capacity; efficiency = actual output ÷ effective capacity; the capacity cushion = 1 − utilisation is the spare capacity.

Measure (simulated, today, average of 20 lunches)	Central CT	Munch Mess	How
Design capacity per lunch	3,600	3,600	20/min × 180 min (bottleneck × window)
Effective capacity per lunch	3,560	3,600	Design × share of time food is available
Actual output (students served)	2,359	599	Simulation
Utilisation	66%	17%	Actual ÷ design
Efficiency	66%	17%	Actual ÷ effective
Capacity cushion	34%	83%	1 − utilisation
Peak load in each mess's own busiest 10 minutes: today → with screens	146% → 107%	38% → 80%	Peak λ ÷ cμ (section 5.3 compares the same 10 minutes, 13:10)
Stations needed at the peak (85% target): today → with screens	4 → 3	1 → 2	ROUNDUP(peak λ ÷ (μ × 0.85))

Averages hide the problem. Over the whole lunch Central's stations look about two-thirds used, with a third spare; in the busiest 10 minutes they get 146% of what they can serve, while Munch's stations run at 38%. Capacity must be planned for the peak. At an 85% planning target (above it waits climb steeply, section 3.3) Central alone would need 4 stations today. Load balancing gets it to 3 without adding any, because Munch has spare stations at the very same minutes. Even perfect balancing leaves the two messes together at about 93% in those 10 minutes (37 arrivals ÷ 40 capacity), so a fifth station or a staggered lunch break (section 6.2) is what would bring the worst 10 minutes under 85%. Workbook: Final_Queue_Capacity_LoadBalancing.xlsx, Capacity and Load Balancing tabs.

4. Where the textbook model breaks at a mess, and what we do instead

M/M/1 or M/M/c assumption	Reality at lunch	How we handle it
Arrivals random at a **constant** average rate	The rate changes through the window, with a peak (a non-stationary process). Its exact timing is not yet measured; the field visit saw full load around 13:18	Apply the formulas **per 10-minute bin**, each with its own λ. The prototype re-estimates λ every 5 s from the last 5 minutes of scans
$\lambda < c\mu$ (stable)	Not true at peak	When $\lambda \geq c\mu$, use a **fluid queue** (below) instead of the steady-state formulas
Service times exponential (cv = 1)	Serving is fairly regular (cv ≈ 0.5–0.6)	M/M formulas then overestimate waits. With one server, the Pollaczek–Khinchine result is exact: Wq scales by $(1 + cv^2)/2$. With c servers the same factor is the Allen–Cunneen approximation. About 0.6–0.7× here. We keep M/M/c as a safe (conservative) estimate
Unlimited waiting space	True for the queue; seats are limited	Seat wait added separately when the hall is full
Servers never stop	A **stock-out** sets μ to 0 until the refill arrives (30 min after the 'running low' call if it has to be made, 10–14 min if already made)	The manager taps 'out' on a star dish; the model sets capacity to 0, adds the remaining refill time and sends students to the other mess
One phase	Face scanner → serving stations → seat	Model the scanner and the serving stations as two phases; add a seat wait

4.1 The fluid queue for overload

During overload, randomness matters less than the simple imbalance between arrivals and capacity. This is the serving-capacity rule: the stations clear 20 students a minute, and anything arriving faster piles up. The queue at the next time step is:

$$B(t + \Delta t) = \max(\, 0,\ B(t) + A(t) - c \cdot \mu \cdot \Delta t \,) \quad \text{(fluid queue)}$$

Each step, the queue grows by the scans that arrived and shrinks by what the counters can serve. If 24 students/min arrive at a hall that serves 20/min, the queue grows by 4/min: after 20 minutes, 80 students are waiting, a 4-minute wait at 20/min.

The wait in a 10-minute slot. A student joining during the slot meets, on average, the line half-way through it, and the stations clear cμ a minute, so wait ≈ (line at the start + line at the end) ÷ 2 ÷ cμ. Worked example (simulated, 13:10–13:20 at Central today): 31 students are still in line from the slot before, 291 join (29.1/min), and the stations can serve 200: 31 + 291 − 200 = 123 people at 13:20. Someone joining mid-slot waits about (31 + 123) ÷ 2 ÷ 20 ≈ 3.8 minutes; someone joining at 13:20 waits 123 ÷ 20 ≈ 6 minutes.

Run-outs reduce capacity. While a star dish is out the stations serve nobody, so in that slot capacity = cμ × (minutes the food was available), not cμ × 10. This is the effective capacity of section 3.5, applied slot by slot.

Does the rule explain the simulation? Applied to each simulated lunch and then averaged, the rule gives Central an average wait of 4.4 minutes today against the simulator's 5.3 (83%). On lunches

without a run-out it is within about 10–25%. The rest is randomness: students arrive in bunches and serving times vary, so short lines form that an average per minute cannot see. That part matters most near 100% load: with the screens Central hovers around 100–107%, and the rule alone gives 0.5 minutes against 1.05 simulated. The workbooks therefore add the textbook M/M/c wait with ρ capped at 95% for the random part (the same rule the live engine uses): line ÷ cμ + M/M/c wait at ρ ≤ 95%.

Why lunch by lunch and not on the average day: a queue piles up more on a bad day than it shrinks on a good one (the 'never below zero' makes it convex), so the rule applied to average arrivals understates the average wait. The dashboard's 'Check the numbers by hand' table applies the rule to one live lunch, block by block, next to the simulator's own count.

4.2 Testing the assumptions with the biometric logs

- **Poisson arrivals?** Within each 10-minute bin, compare the mean and standard deviation of the gaps between scans. For Poisson arrivals they are roughly equal (cv ≈ 1).
- **Peak shape:** plot arrivals per 10 minutes for each weekday. This gives λ(t), the main input to every formula.
- **Little's Law check:** L = λ × W against photo headcounts.
- **Service rate μ:** time 20–30 students at one serving station with a stopwatch, or count trays issued per minute at the tray counter.

5. How the simulation applies queuing theory

The prototype has two halves. A **simulator** plays the role of reality: made-up students, real queues, real seats, real food stock. An **engine** plays the role of the live system: it sees **only** what the real deployment would see, namely biometric entry scans and the mess manager's food-status taps ('running low', 'out', 'refilled'). Keeping them separate means the simulation tests whether the estimates are any good, not just whether the formulas are.

To compare outcomes fairly, every lunch is simulated **twice with exactly the same students**: once with no advice (today), once with screens outside each mess.

5.1 The simulator (ground truth)

Element	How it is modelled	Queuing concept
Arrivals	Random arrivals whose average rate is the department lunch breaks added up (section 5.1, Arrival curve); each student starts at their habitual mess (h_i)	Non-homogeneous Poisson arrival process, λ(t)
Stage 1: scan and plate	1 face scanner per mess, 2 s per student (assumed), then collecting a plate (scan + plate = 30–45 s, group estimate). Students who cannot scan yet wait OUTSIDE, unseen by the system	Single-channel first phase
Stage 2: food	c = 2 stations per mess (measured); service time random with mean 6 s (assumed) and cv 0.6; then the rest of food collection (collecting food takes 1–2 min in all without a queue, group estimate)	Multiple-channel service, general service times (M/G/c)
Queue discipline	First come, first served	FCFS
Food stock	Batches of the main dish (900 at Central, 500 at Munch, assumed); when stock falls to a call level (assumed) the mess calls the kitchen; the refill arrives 30 min later if it has to be made, or 10–14 min later if already made (both measured; the group estimates it is already made about 70% of the time); if stock hits 0, the stations stop	Service interruption: μ = 0 during a stock-out
Stage 3: eat	900 (Central) and 492 (Munch) seats; if full, the student waits; eating takes 14–18 min (group estimate)	Phase with finite capacity
Stage 4: dispose	Leftovers and plate, 1–2 min (group: 2 min at most); the seat is already free	Time in system W = sum of the four stages
What we measure	Queue wait = time queueing for the scanner, a station and a seat. Throughput = door to leaving (group: 17–35 min). Simulated: ~19.8 min for students who meet no queue, ~25.6 min around today's 13:17 peak	Wq and W
Screens	At the door, a student is advised to stay or walk 30 s to the other mess; κ = 60% follow	Routing between two parallel queues

Arrival curve: department lunch breaks added up

Students do not arrive on one smooth curve: each department breaks for lunch at its own time. We build the arrival rate λ(t) from the timetable the group collected. Department sizes are rough counts that add up to 3,150, more than the 2,960 plates, so they are used as **shares** and scaled to 2,960.

Department	Lunch break	Students (rough)	Status
MIT	12:00–13:00, or 13:00–14:00 on	1,100	Timetable and size from the group; lab-day

Department	Lunch break	Students (rough)	Status
	a lab day		share 45% assumed (labs 3 days/week for electrical, 2 for others)
TAPMI	13:05–14:00	850	Group
DOC/MICFE	12:30–13:30	550	Group
SMI	12:00–13:00	450	Group
DLHS/MILHS	12:40–13:40	~100	Timing from the group; size assumed
MLS	12:50–13:50	~50	MLS + MIRM ~100 (group); half each assumed
MIRM	not known (12:30–13:30 assumed)	~50	Assumed

$$\lambda(t) = \Sigma_d\ n_d \cdot f_d(t), \quad f_d = \text{Normal}(\text{start_d} + 20, 20) \text{ cut off to } [\text{start_d}, \text{end_d}] \quad \text{(arrival rate)}$$

Each department starts arriving when its break starts; a student's lag after the bell is Normal(20, 20) minutes, cut off at the end of the break. The lag is **calibrated, not measured**: with it, the no-screen simulation gives a 20–26 minute visit and a ~14 minute 90th-percentile wait, close to the group's 17–35 minutes and the ~15 minutes seen in Phase 1; a tighter lag (15 ± 10, everyone rushing at the bell) gives ~30-minute peak visits and a 23-minute 90th percentile, above what was observed. The curve has two surges: about **12:10** (MIT and SMI) and a larger one at **13:10–13:30** (MIT's lab-day batch, TAPMI, DOC/MICFE, DLHS/MILHS and MLS overlap). The busiest 10 minutes start at 13:10, matching the full hall photographed at 13:18.

What this adds to the decision: staggering one department's break by 15 minutes cuts today's average wait from 4.3 to 3.3 minutes (SMI to 11:45) or 1.7 minutes (TAPMI to 13:20) with no screens at all. That makes alternative A7 (stagger class end times) concrete, though it needs academic administration; the screens get further (0.8 minutes) without changing any timetable.

5.2 The engine (what the live system estimates from scans)

Step 1: arrival rate. $\hat{\lambda}$ = scans in the last 5 minutes ÷ 5. This is the measured λ.

Step 2: the queue. The backlog B(t) follows the fluid-queue formula from section 4.1, with capacity cμ (0 if the manager has reported a star dish 'out').

Step 3: people inside, using Little's Law. There are no exit scans, so we cannot count people leaving. Instead:

$$\hat{L}(t) = B(t) + S(t), \quad \text{where } S(t) = \text{students served in the last 19 minutes (rest of food collection + eating + disposal)} \quad \text{(people inside)}$$

On a normal day this gives L ≈ λ × W, Little's Law with the normal visit. In a surge, students stuck in the queue still count as inside, so the visit stretches automatically. An earlier version counted 'scans in the last 18 minutes', which forgot students in long queues. Switching to this method cut the error in 'people inside' from about **34 to 11 people**.

Step 4: predicted time to food for a student deciding now. Every term is a queuing quantity:

$$\text{Total} = \text{walk} + \text{queue wait} + \text{service time } (1/\mu) + \text{seat wait} + \text{food delay} \quad \text{(prediction)}$$

- **Queue wait** = (students ahead ÷ cμ) + growth during overload + M/M/c Wq for the remaining randomness. Students ahead = B(t) projected forward by the walk time. Growth = max(0, λ − cμ) × H/2, i.e. the queue's average extra growth over the next H minutes (the fluid model). The

M/M/c term uses Erlang C, with ρ capped at 95% because the formula blows up at 100% and the fluid term handles overload.

- **Seat wait** = extra people beyond N ÷ seat turnover (N / eating time seats freed per minute). Zero unless the hall is full.
- **Food delay**, weighted by the priority matrix (section 7.4). A **star dish** out (any star dish) is an outage: nobody is served until the refill. For any other tray out, or any tray running low, a student is delayed only if they would reach the counter **after the tray empties and before its refill**: delay = w × (R − t), where t is when they reach the counter and R the refill time. 'Running low' is tapped when the mess calls the kitchen, with ~20 minutes of food left (protocol), so a student served within 20 minutes loses nothing. Weights w: star dishes 1.0, roti 0.5, dessert and curd/raita 0.3, base food 0.1; table items are not tracked.
- **The refill clock runs from the kitchen call.** At the call the expected refill is 0.7 × 12 + 0.3 × 30 ≈ 17 min. As minutes pass without a refill, the estimate is updated: after 15 minutes the 10–14 minute 'already made' case is ruled out, so about 15 more minutes remain. Once the refill must have arrived, serving is assumed to resume even if the manager forgets 'refilled'.

Step 5: expected time inside for someone arriving now = scan and plate + queue wait + serving + food line + eating + disposal. It is about 20 minutes on a normal day (the group's estimate; the stage timings' mid-points add up to ≈ 19.5) and rises during surges. The manager dashboard shows it, next to a 'When students arrive' panel: the arrival curve from the department lunch breaks, the arrivals so far, the 40/min serving capacity of both messes, and which departments are on their break now.

5.3 Turning predictions into advice (routing)

If every screen said 'go to the emptier mess', everyone would go there and it would become the busy one (**herding**). Instead the engine finds the split of arriving students p_i that makes the predicted waits **equal** in both messes (a Wardrop equilibrium, solved by bisection). Then, at the busier mess's door:

$$\text{share of students asked to switch} = (h_i - p_i) / (h_i \cdot \kappa) \quad \text{(door switch rate)}$$

…but only if switching saves at least Δ_min = 2 minutes after the 30-second walk. The screen shows a strong message ('Munch Mess is faster right now →') when that share is high, a soft one ('In a hurry? …') when it is moderate, and 'Come on in' otherwise.

In capacity terms this is load balancing: parallel resources should run at equal utilisation, and with equal capacity the balanced level is total demand ÷ total capacity. In the busiest 10 minutes that level is 93%. Today Central runs at 147% and Munch at 38%; with the screens 106% and 78%, so the gap between them falls from 108 to 29 percentage points (workbook: Load Balancing, section 9; the dashboard shows the same as bars). The screens stop short of perfect balance because only 60% follow and they only advise a switch that saves at least 2 minutes. With equal capacities, equal predicted waits (the Wardrop split) and equal utilisation are the same target. Section 3.3 shows why this matters so much: the last few percent of utilisation cost the most waiting.

6. Results from the simulation

> **Simulated, not measured.** 20 simulated lunches per row, the same students in every scenario (paired comparison); ± is a 95% confidence interval across lunches. Measured: 2,960 students, the 80/20 habit split, seats, 2 stations and 1 scanner per mess, refill times. Group estimates: scan + plate 30–45 s, collecting food 1–2 min plus the queue, eating 14–18 min, disposal 1–2 min, food ready 70% of the time. **Assumed:** 10 students/min per station, scan and plate times, department sizes (rough). **Calibrated:** the arrival lag after each lunch break. Every number is reproduced by `python -m messq.scenarios`.

6.1 Four scenarios on a busy day (60% follow the screens)

Scenario	Queue wait (min)	90th pct wait	Door to exit (min)	Stations busy C / M	Run-outs	Moved	Moved, worse off
1 Current system	4.31 ± 0.63	13.9	23.9	66% / 16%	0.5	0	–
2 Screens: queue information only	1.19 ± 0.33	3.1	20.8	59% / 23%	0.5	239	2%
3 Screens + food status (proposal)	0.80 ± 0.10	2.1	20.4	58% / 24%	0.3	261	2%
Naive: go to the shorter queue	0.76 ± 0.10	2.0	20.4	58% / 24%	0.5	271	5%

Queue wait = time queueing for the scanner, a serving station and a seat. Throughput = door to leaving (all four stages). An **unsuccessful redirect** is a redirected student whose wait plus walk exceeded the average wait of students who stayed at the mess they left and arrived within 2 minutes.

- The proposal cuts the average queue wait from **4.3 to 0.8 minutes** and the 90th percentile from **13.9 to 2.1 minutes**; throughput falls from 23.9 to 20.4 minutes.
- **Food status helps**: queue information alone gives 1.2 minutes; adding the manager's food status gives 0.8. The 13:10 surge drains the trays just when the queue is longest, so knowing about a run-out matters.
- About **1 in 11 students** walks to the other mess (261 of ~2,960). The LP optimum (section 7.6) moves 261, so the live system matches the theoretical split.
- Without screens the simulated 90th-percentile wait (~14 min) is close to the ~15 min observed in Phase 1, and the simulated throughput (~19.8 min with no queue, ~25.6 min at the peak) sits in the group's 17–35 min range. This is partly by construction: the arrival lag was calibrated to it.

6.2 Stress tests: does the benefit persist? (average queue wait, minutes)

Changed input	1 Current	2 Queue only	3 Proposal
Demand +20% (3,552 students)	12.05	2.17	2.02
Students rush at the bell (lag 15 +/- 10 min, not 20 +/- 20)	8.52	2.05	1.68
Heavy lab day: 70% of MIT eat 13:00-14:00	5.96	1.41	1.34
No MIT labs: all MIT eat 12:00-13:00	8.78	1.20	0.95
Stagger: SMI lunch 15 min earlier (11:45-12:45)	3.27	0.69	0.56
Stagger: TAPMI lunch 15 min later (13:20-14:15)	1.65	0.85	0.55
One bell curve (old assumption: peak 13:05, spread 35)	5.76	1.57	0.88

Changed input	1 Current	2 Queue only	3 Proposal
Only 30% follow the screens	4.31	1.18	1.10
Everyone follows the screens	4.31	1.16	0.76
Slower serving: 8 students/min per station	12.88	1.83	1.86
Faster serving: 12 students/min per station	1.79	1.03	0.56
More stock-outs: next batch never ready	6.52	2.56	0.94
No stock-outs: next batch always ready	3.61	0.78	0.76
Slower eating: 17-21 min	4.31	1.19	0.80

The proposal cuts the wait in every stress test (by about 65–90%) and beats or ties queue information only in every one. With no stock-outs the food signal adds nothing (2 and 3 converge); with more stock-outs it matters most. The department tests show the timetable matters as much as any operational input: a day without MIT labs, when MIT and SMI break together at 12:00, is twice as bad as an average day; moving TAPMI 15 minutes later more than halves the wait; the old single bell curve gave similar averages but hid the two surges.

6.3 Why the spreadsheet and the simulator differ (reconciliation)

The workbook's S15 tab gives 3.57 → 0.57 minutes. It carries the queue from slot to slot (the fluid queue), charges each student the line half-way through their slot, and adds the M/M/c wait at $\rho \leq 95\%$ for randomness. We started the simulator with the spreadsheet's simplifications and added one real-world feature at a time:

Simulator step	Without screens	With screens
1 Spreadsheet-like: exponential serving, no run-outs, no scanner or seat limits	3.77	0.84
2 + realistic serving variability (cv 0.6)	3.50	0.66
3 + one face scanner per mess (2 s each)	3.62	0.75
4 + seat limits	3.62	0.75
5 + food run-outs and refills (full model)	4.31	0.80

Under the same assumptions the two models agree without screens within about 5% (3.57 vs 3.77). The small gap is randomness that a slot-by-slot calculation smooths over. With screens the spreadsheet is optimistic because it routes with perfect knowledge every 10 minutes; the simulator routes on live estimates with a delay and the 2-minute rule. The headline numbers come from the full simulator because it includes randomness, live estimation, the scanner, seats and run-outs.

6.4 What we fixed on the way, and the limits

- **A simulator bug understated run-outs**: after a refill, the waiting queue was served with start times in the past. Fixed; run-outs now cost the time they should.
- **Food status was first used the wrong way**: a flat penalty for 'running low' sent students away even when they would be served long before the tray emptied (36% unsuccessful redirects). Charging the delay only to students who would reach an empty tray, and counting the refill from the call, fixed it.
- **Limits**: the habit split and the service rate are assumed; department sizes are rough and the arrival lag is calibrated, not measured; the bias explanations are hypotheses; the simulated visit for a student who meets no queue (19.8 min) is a little above the ~18 minutes measured

in Phase 1, so the group's stage timings may be slightly long. The biometric logs and a few stopwatch timings settle these (workbook: Data Collection).

7. The other MDS course topics applied to the project

The course session plan covers more than queuing. Each topic has a natural place in this project:

Session	Topic	Where it applies in our project
2	Nature of business decisions	Lunch allocation is a repeated, structured operational decision taken under time pressure
3	Problem identification, structuring, root cause	Separating symptoms (queues, run-outs) from causes (peak arrivals, reactive replenishment, buffer sizing)
4	Decision frameworks, biases, ethics	Habit and herding in students' choices; privacy and honesty in the system's design
5–6	Decision dilemma and priority matrix	The ABC/XYZ menu matrix: which items deserve forecasting effort
7	Decision trees; risk and uncertainty	Whether to send an extra batch before the peak or wait and react
10–11	Linear programming	Splitting a limited batch of a star dish across centres
12–14	Integer programming, branch and bound	How many serving stations to open; when to schedule replenishment trips
15	Queuing theory	Sections 2–6 of this document
16–17	MCDM: AHP, BWM, ISM (MICMAC)	Weighting service level vs waste vs stock-out risk; mapping how the causes drive each other
18	Heuristics and metaheuristics	The routing rules in the prototype; searching for a good replenishment schedule with the simulator

7.1 Nature of business decisions (Session 2)

The allocation of food and staff at lunch is a **structured, recurring operational decision**: the same choice is made every day across five centres, with good data available in principle. That makes it a strong candidate for a decision-science model rather than ad-hoc judgement under pressure, which is what we observed.

7.2 Problem identification and root cause analysis (Session 3)

A '5 Whys' chain from our field evidence:

1. Why do students wait ~15 minutes? Because the serving station runs at or above capacity at peak.
2. Why is it above capacity? Because arrivals cluster at peak times (timing to be confirmed by the logs), and service stops when a dish runs out.
3. Why do dishes run out? Because replenishment is triggered only when stock is running low, and then takes 30 min if the kitchen has to make it (10–14 min if already made).
4. Why is it triggered late? Because there is no live view of demand: the kitchen reacts to empty wells.
5. Why is there no live view? Because entry data (biometric scans) isn't used for operations.

Root causes for a fishbone diagram: **arrival concentration** (people), **serving rate and refill-at-counter** (process), **reactive replenishment and buffer sizing** (policy), **no live demand data** (information), and **one shared delivery vehicle path** (logistics).

7.3 Decision frameworks, biases and ethics (Session 4)

- **Status-quo / habit bias:** students return to the same mess (about 80/20, measured) even when the other is quicker. The screen design uses defaults, concrete gains ('Enough time for a chai') and fresh-food cues to counter this.
- **Herding:** if everyone follows the same signal, the signal fails. Hence the equilibrium split rather than 'everyone go to the emptier one'.
- **Anchoring in planning:** the kitchen plans from the published menu; our field visit found the served sweet differed from the published one, so forecasts anchored on the published cycle will drift.
- **Ethics:** biometric logs are personal data under India's DPDP Act 2023. We request only timestamps and location, never face data, pseudonymise IDs, report only aggregates, and never show misleading advice on screens.

7.4 Decision dilemma and priority matrix (Sessions 5–6)

Our ABC/XYZ matrix classifies each menu entry for the week of 31 Aug–6 Sep by **criticality** (cost of getting the quantity wrong) and **predictability** (how stable demand is):

Policy	Class	Menu entries	What to do
Tight Control	AZ (critical, erratic)	28	Measure and model: star dishes, chapattis, curd and raitha
Calculated Issue	AX (critical, stable)	9	Derive from plate count: one-per-plate sweets (and Sunday's portioned chicken)
Buffered Issue	CZ (cheap, erratic)	40	Issue with a safety margin: rice, dal, sambar, rasam, dry veg
Free Issue	CX (cheap, stable)	26	Issue without control: pickle, salt, sugar, fryums

Counts are menu entries for the week (MDS_dataset.xlsx, Priority Matrix sheet). The matrix decides where the queuing and optimisation effort goes: the **AZ items**. The field visit confirmed the order: star dishes ran out first (13:20–13:23), the curd later (13:39). (The dataset's field notes label the curd AX, while its menu ratings give it AZ; we follow the ratings.)

The matrix also sets how much each tray matters to the live system. The mess manager marks trays running low, out or refilled, and each tray's weight decides how strongly that moves students:

Tray	Item type	Class	Weight	Effect when OUT
Gravy sabji, special rice	Star dish	AZ	1.0	That mess is closed for routing: students are sent to the other mess
Roti	Made fresh in batches	AZ	0.5	Adds 0.5 × E[refill] ≈ 9 min to that mess's wait
Dessert	One per plate	AX	0.3	Adds ≈ 5 min
Curd / raita	Fresh side	AZ	0.3	Adds ≈ 5 min
Rice, dal, dry sabji, sambar / rasam	Base food	CZ	0.1	Adds ≈ 2 min (interchangeable; 'quietly absorbed')
Salt, pickle, fryums	Table item	CX	0	Not tracked

'Running low' adds a quarter of the 'out' effect (assumed), e.g. a star dish running low adds ≈ 4 min. The weights are the group's reading of the matrix: class A above class C; within A, the dish students come for above items made continuously (chapati), one-per-plate sweets and fresh sides. They should be confirmed with the CT manager.

7.5 Decision trees under risk (Session 7)

Decision: at 12:30 (before the peak), should the kitchen cook the next batch of the star dish **ahead** (so a refill reaches the mess 10–14 min after the call), or **wait and react** to the "running low" call (30 min)? (All costs and probabilities below are **illustrative**.)

- Chance of a high-demand day: P(high) = 0.4; normal day: 0.6.
- Pre-emptive batch, normal day: 40 portions wasted at ₹30 = **₹1,200**. High day: no shortage, **₹0**.
- Wait and react, high day: a ~25-minute run-out delays ~150 students by ~10 min; at a shadow cost of ₹5 per student-minute = **₹7,500**. Normal day: **₹0**.

$$\text{EMV(pre-emptive)} = 0.6 \times 1{,}200 = ₹720 \qquad \text{EMV(react)} = 0.4 \times 7{,}500 = ₹3{,}000 \quad \text{(expected cost)}$$

Pre-staging has the lower expected cost. The **break-even probability** is p = 1,200 / (1,200 + 7,500) ≈ **0.14**: if a high-demand day is more likely than about 1 in 7, pre-stage. The biometric logs supply this probability: how often lunch arrivals exceed the planned count.

7.6 Linear programming (Sessions 10–11): the primary decision model

Primary model: how many students to direct to each mess in each 10-minute slot. This is the managerial decision the project supports, so it is the main LP. Food is an operational constraint inside it.

- **Decision variables:** y_t = students redirected from Central to Munch in slot t; QA_t, QB_t = students still queueing at Central and Munch at the end of slot t.
- **Data:** D_t = arrivals in slot t (the S15 arrival curve); h = share heading to Central by habit (80%, measured); κ = share who follow the screen (60%); cap = stations × service rate (20/min per mess); walk = 0.5 min; F_B = Munch's star-dish portions for the lunch.

$$\text{Minimise} \;\; 10 \cdot \Sigma_t (QA_t + QB_t) \; + \; \text{walk} \cdot \Sigma_t \, y_t \quad \text{(student-minutes of waiting and walking)} \quad \text{(objective)}$$

$$\text{s.t.} \;\; QA_t \geq QA_{t-1} + h \cdot D_t - y_t - 10 \cdot \text{capA}; \quad QB_t \geq QB_{t-1} + (1-h) \cdot D_t + y_t - 10 \cdot \text{capB}; \quad 0 \leq y_t \leq \kappa \cdot h \cdot D_t; \quad \Sigma_t \, [(1-h) \cdot D_t + y_t] \leq F_B; \quad Q \geq 0 \quad \text{(constraints)}$$

The queue constraints are the fluid queue from section 4.1 written as linear inequalities, so the LP is exact for the fluid model. Solved with the simplex method (SciPy/HiGHS; Excel Solver gives the same): **today 9,087 student-minutes (3.1 min per student) → 131 (0.04 min)**, by redirecting **261 students (9%)** at the peaks. The simulator's proposal redirects about 261, the same as this optimum. The LP is a lower bound: it has no randomness and perfect foresight.

Shadow price of food. Munch needs about 860 star-dish portions to carry the optimal flow (about 600 for its habitual diners alone). Below that, the food constraint binds: with 800 portions each extra portion saves about 20 student-minutes, with 700 about 40. Routing students and allocating food are one decision, not two.

Secondary model: allocating the star dish to support the flow. x1, x2 = portions ready at Central and Munch before the peak; maximise $\Sigma \, v_i \cdot x_i$, where v_i is the number of students per minute who would meet an empty tray at mess i under the flow plan; subject to x1 + x2 ≤ K (portions the kitchen can have ready) and $x_i \leq d_i$ (that mess's demand under the flow plan). With two variables it is solved by the **graphical method** (evaluate the corner points); the busier mess is filled first.

7.7 Integer programming and branch and bound (Sessions 12–14)

Some decisions can only take whole-number values, so they need integer programming:

- **Number of serving stations** at each mess and time slot: $c_{i,t} \in \{1, 2, 3, \ldots\}$ (today 2 per mess). Minimise staff cost Σ cost $\cdot\ c_{i,t}$ subject to the queuing constraint $\lambda_{i,t} \le \rho_{max} \cdot \mu \cdot c_{i,t}$ and a total staff limit. The LP answer (for example 2.6 stations) is not usable; branch and bound finds the best integer staffing.
- **Replenishment trips:** $y_t \in \{0, 1\} = 1$ if the vehicle leaves at time t. Each trip carries at most Q portions, arrives 30 min after a call if it must be made (10–14 min if already made), and cumulative deliveries must stay ahead of cumulative forecast demand. Minimise trips plus shortage cost.

7.8 Multi-criteria decision making: AHP, BWM, ISM (Sessions 16–17)

Our deck proposes a **weighted decision table** with four criteria: service level, stock-out risk, waste and utilisation. AHP turns judgements into weights. **Illustrative** pairwise comparisons (1 = equal, 3 = moderately more important, 5 = strongly more important):

	Service level	Stock-out risk	Waste	Utilisation	Weight
Service level	1	1	3	5	**0.39**
Stock-out risk	1	1	3	5	**0.39**
Waste	1/3	1/3	1	2	**0.14**
Utilisation	1/5	1/5	1/2	1	**0.08**

Consistency ratio ≈ 0.002 (well below the 0.10 limit), so these judgements are consistent. The weights then score alternatives such as: A1 status quo; A2 a 3rd serving station during the peak (timing set from the logs); A3 pre-staged replenishment of AZ items; A4 door screens and routing; A5 staggered class end times. Queuing results (section 3) supply the service-level score of each alternative.

BWM (Best–Worst Method) gives the same kind of weights with fewer comparisons: pick the most important criterion (for example service level) and the least (utilisation), and compare each criterion only against those two.

ISM with MICMAC maps how the causes drive each other. Candidate factors: peak arrival concentration, serving rate, reactive replenishment, buffer sizing, menu substitution, no live demand data, delivery path congestion, queue length, stock-outs and student waiting time. MICMAC then classifies them as drivers (high driving power: arrival concentration, no live data), linkage factors (replenishment timing), or outcomes (waiting time, stock-outs). This tells us which levers to pull first.

7.9 Heuristics and metaheuristics (Session 18)

- The door-screen rule **'suggest switching only if it saves at least 2 minutes'** is a heuristic. In the simulation it performs about as well as the full equilibrium calculation at the doors, because the feedback loop (a 30-second walk) is fast.
- The equilibrium split is found by **bisection**, a simple numerical search.
- Choosing a replenishment schedule that minimises waiting plus waste across a whole lunch has no clean formula because of the queues. A **metaheuristic** (simulated annealing or a genetic algorithm) can search over schedules, scoring each one by running our simulator: **simulation-optimisation**.

8. What we have measured and what we still assume

Input	Value used	Status	How to get it
Seats	Central 900, Munch 492	**Measured**	Done
Normal visit (scan to leaving)	≈ 18 min	**Measured** (approx.)	Confirm with a few timed visits
Plates per lunch	≈ 2,960	**Measured**	Done (Phase 1)
Replenishment lead time	30 min after the call if it has to be made; 10–14 min if already made	**Measured**	Done
Chance the next batch is already made	≈ 70%	**Group estimate**	Confirm with the CT kitchen
Arrival rate λ(t)	Department lunch breaks (timetable, group); sizes rough; lag after the bell calibrated	Timetable measured; rest assumed	**Biometric logs** (requested)
Students per lunch	2,960 (= plates)	**Measured**	Done (Phase 1)
Lunch window	11:30–14:30	**Measured**	Done
Serving stations c	2 per mess	**Measured**	Done
Face scanners	1 per mess	**Measured**	Done
Service rate μ per station	10 students/min (6 s)	Assumed	**Stopwatch** 20–30 students at peak
Scan time	2 s per student	Assumed	Stopwatch at the scanner, or gaps between scans in the logs
Habit split between messes	80% Central / 20% Munch (~2,360 / ~600)	Measured (group)	Biometric logs confirm it (a no-screen baseline)
Compliance κ	60%	Assumed	Pilot
Walk between doors	30 s	Assumed	Walk it with a stopwatch
Food batch and reorder level	Central 900 (reorder at 500), Munch 500 (reorder at 300)	Assumed	CT kitchen
Throughput (door to leaving)	17–35 min depending on the rush	**Group estimate**	Time 20 students door to exit
Scan + collect a plate	30–45 s (2 s of it the face scan)	**Group estimate**	Stopwatch at the entrance
Collect food (no queue)	1–2 min (about 6 s of it at the serving station)	**Group estimate**	Stopwatch
Eating	14–18 min	**Group estimate**	Time 20 students
Disposal	1–2 min (≤ 2)	**Group estimate**	Observe the return station
Food left at a 'running low' tap	~20 min	Assumed (protocol)	Record tap and run-out times
Tray weights	Star 1.0, roti 0.5, dessert / curd 0.3, base food 0.1	Group judgement from the ABC/XYZ matrix	Confirm with the CT manager

Input	Value used	Status	How to get it
'Running low' share	0.25 of a run-out	Assumed	Logs of how long trays last after 'running low'

The most valuable next measurement is **μ, the serving rate**. Every waiting-time formula depends on it, and section 3.3 shows that small errors near full utilisation produce large errors in predicted waits.

Appendix A: formula sheet

Quantity	Formula	Notes
Utilisation	$\rho = \lambda / (c \cdot \mu)$	Stable only if $\rho < 1$
Little's Law	$L = \lambda \cdot W$, $Lq = \lambda \cdot Wq$	Holds for any stable system
M/M/1 wait in queue	$Wq = \lambda / (\mu(\mu - \lambda))$	One server
M/M/1 time in system	$W = 1 / (\mu - \lambda)$	
M/M/1 number waiting	$Lq = \lambda^2 / (\mu(\mu - \lambda))$	$= \lambda \cdot Wq$
Offered load	$a = \lambda / \mu$	
Erlang C	C(c,a) = [(a^c/c!)·c/(c−a)] / [Σ_{k<c} a^k/k! + (a^c/c!)·c/(c−a)]	P(an arrival waits)
M/M/c wait in queue	$Wq = C(c,a) / (c \cdot \mu - \lambda)$	
M/G/1 correction	$Wq \approx Wq(M/M/1) \cdot (1 + cv^2) / 2$	Pollaczek–Khinchine (exact for one server); cv of service times
Fluid queue (overload)	$B(t+\Delta t) = \max(0, B(t) + A(t) - c \cdot \mu \cdot \Delta t)$	Used when $\lambda \geq c\mu$
People inside (prototype)	$\hat{L} = B(t) + S(t)$	S = served in last T_eat min
Seat capacity limit	$\lambda_max = N / W$	From Little's Law
Door switch share	$(h_i - p_i) / (h_i \cdot \kappa)$, if saving ≥ Δ_min	Routing at the busier mess
Expected monetary value	EMV = Σ P(outcome) · value(outcome)	Decision trees
M/G/c correction	$Wq \approx Wq(M/M/c) \cdot (1 + cv^2) / 2$	Allen–Cunneen approximation (c servers)
Wait in a 10-min slot	(B_start + B_end) / 2 / (c·μ) + Wq(M/M/c at ρ ≤ 95%)	Fluid line + randomness
Capacity with a run-out	c·μ × (10 − minutes food out)	Effective capacity per slot
Utilisation, efficiency	actual ÷ design; actual ÷ effective	Capacity management
Capacity cushion	1 − utilisation	Spare capacity
Stations needed	ROUNDUP(peak λ ÷ (μ × ρ_target))	ρ_target = 85%
Balanced load	$\rho^* = (\lambda_A + \lambda_B) \div (c_A \cdot \mu + c_B \cdot \mu)$	Load balancing target

Appendix B: where each idea lives in the code

The prototype is in the GitHub repository `mrnotamazing/SmartScan_Mess` (branch `claude/laughing-bell-r2ub69`).

Idea	File	What it does
Erlang C, M/M/c waiting time	`messq/queueing.py`	erlang_c(), mmc_wait()
Arrival rate, fluid queue, people inside	`messq/estimator.py`	λ from scans; B(t); S(t); $\hat{L} = B + S$
Predicted time to food	`messq/waits.py`	walk + queue + service + seat + food
Equilibrium split, door switch shares	`messq/routing.py`	Wardrop split by bisection; switch rule with 2-min threshold
Live engine	`messq/engine.py`	Combines scans, the manager's food-

Idea	File	What it does
		status taps (weighted), estimates and routing
Tray weights	`config/default.toml` [[tray]]	Item type, ABC/XYZ class and weight for each tray
Simulator (ground truth)	`messq/simulation.py`	Four stages: scan + plate, serving station + food line, eating, disposal; food stock and refills
Scenarios, payoff matrix, reconciliation	`messq/scenarios.py`	Writes results/*.json (20 lunches per row, same students)
Decision workbook	`tools/build_workbook.py`	Builds docs/Final_MDS_Decision_Workbook.xlsx from the Inputs and results; LP solved with SciPy
All tunable numbers	`config/default.toml`	Measured and assumed values, each labelled
Monte Carlo comparison	`messq/simulate.py`	Runs many lunches; prints the tables in section 6
Department arrival curve	`config/default.toml` [[simulation.slot.group]]; `messq/config.py`	Lunch break, rough size and lab-day share per department; λ(t) = the breaks added up
Queue, capacity and load-balancing workbook	`tools/queue_capacity_data.py`, `tools/build_queue_capacity_workbook.py`	docs/Final_Final_Queue_Capacity_LoadBalancing.xlsx: M/M/1 and M/M/c per process, Little's Law per slot, seat turnover, static vs dynamic routing
Live demo dashboard	`messq/demo/`	Same lunch twice (without / with screens); 'When students arrive' panel shows the timetable curve, arrivals so far, capacity and who is on break

Appendix C: what changed in this final version

We re-checked every calculation against queuing theory, capacity management and load balancing. What we corrected:

- **Service rate in the M/M/1 example.** The M/M/1 example and the 'waits explode' table used μ = 240/hour (15 s a student); everything else in the project uses 10/min (6 s). They now use 10/min, so the numbers match the workbooks.
- **Reading the 15-minute delay.** The 15-minute delay was 'read backwards' with a steady-state M/M/1 formula. A wait that long cannot come from a steady state at our service rate; it is an overload. Section 3.3 now reads it with the fluid queue (about 300 people ahead at 20 served a minute).
- **Seats vs people inside.** Seats were compared with everyone inside the hall, queue included. Seats only hold served students, and the stations cap that flow at 20 a minute, so at most ≈ 320 seats per mess are ever in use. Section 3.2 and the OM table now say so.
- **Line carried between slots.** The per-slot M/M/c and M/M/1 tables in the queue workbook ignored the line carried over from an overloaded slot, so after 13:30 they showed short waits while ~200 students were still waiting. They now carry the line from slot to slot (fluid queue) and take run-out minutes off capacity (effective capacity).
- **S15 queue wait.** S15 in the decision workbook charged each student the line at the end of their slot and, in overloaded slots, added the steady-state wait on top of it. It now charges the line half-way through the slot plus the M/M/c wait at ρ ≤ 95% (the same rule as the live engine). Result: 3.57 → 0.57 minutes (was 3.52 → 0.47), within 5% of the simulator under the same assumptions.
- **Name of the cv correction.** The $(1 + cv^2)/2$ correction is exact (Pollaczek–Khinchine) only for one server; for two stations it is the Allen–Cunneen approximation. Labels corrected.
- **Capacity and load-balancing measures.** Added the missing capacity-management measures (design and effective capacity, utilisation, efficiency, capacity cushion, stations needed at an 85% target) and the load-balancing measure (utilisation at each mess against the balanced level), section 3.5 and 5.3.
- **Stale numbers.** Removed stale numbers from an older model version (a 7.2 → 8.0 minute wait; a 16.7-minute no-queue visit; a 17–18 minute quiet visit).